UPES, Dehradun
School of Computer Science · Artificial Intelligence Cluster

AdaGrad (Adaptive Gradient Algorithm)

Mid-Semester Assignment Report

Course: Applied Machine Learning (CSAI2017P)
Student: Adit Kishor Nautiyal
SAP ID: 590024518
Batch: B15
Semester: III (Aug–Dec 2026)
Submitted to: Kunwar Siddharth, Assistant Professor
Date: October 17, 2026

Scan to open the animation / code

Abstract

AdaGrad (Adaptive Gradient Algorithm) is a first-order optimization method designed to adjust learning rates independently for individual model parameters. Unlike conventional gradient descent, which typically applies a common learning rate, AdaGrad accumulates the squared gradients of each parameter and uses this history to scale subsequent updates. This mechanism can be particularly useful in learning problems involving sparse features or substantially different gradient magnitudes. This project investigates AdaGrad using a two-dimensional quadratic objective function with unequal curvature along its coordinate axes. A Python-based interactive visualization is developed to illustrate the parameter trajectory, objective value, accumulated squared gradients, and coordinate-wise effective learning rates across successive iterations. The experiment is intended to examine how gradient history influences update magnitudes and optimization behaviour. A key limitation of AdaGrad is that its accumulated squared gradients never decrease, causing effective learning rates to shrink over time. Consequently, parameter updates may become excessively small, potentially slowing further optimization progress. The analysis evaluates these characteristics using the numerical trace and graphical outputs generated by the implementation.

Contents

1 Introduction **2**

2 Intuition and Core Idea **2**

3 Mathematical Formulation **3**

4 Algorithm / Procedure **4**

5 Worked Example (Step by Step) **4**

6 Animation / Software Demonstration **5**

7 Analysis **7**
7.1 Assumptions 7

8 Applications **7**
8.1 Natural Language Processing 7
8.2 Online Learning and Recommendation Systems 7
8.3 Machine Learning with Sparse Features 8
8.4 Summary 8

9 Conclusion **8**

1 Introduction

Optimization is a fundamental process in machine learning that involves finding model parameters that minimize an objective function, typically representing prediction error. Gradient-based methods perform this optimization iteratively by adjusting parameters according to the gradients of the objective function. However, selecting an appropriate learning rate is challenging when different parameters have gradients with substantially different magnitudes. A single global learning rate may produce excessively large updates for some parameters while causing slow progress for others.

AdaGrad (Adaptive Gradient Algorithm) addresses this limitation by accumulating the squared gradients independently for each parameter and using these accumulated values to scale subsequent updates. Consequently, parameters with larger accumulated gradient magnitudes receive smaller effective learning rates, while parameters with smaller accumulated values can retain comparatively larger learning rates. This property can be beneficial in sparse-feature applications such as text classification, where certain words appear frequently while others occur infrequently but may provide useful classification information. AdaGrad was developed as an adaptive subgradient method for online learning and stochastic optimization [1].

This project investigates AdaGrad using a two-dimensional convex quadratic objective function with different curvature along its coordinate axes. A Python-based visualization is used to examine the optimization trajectory, changes in the objective value, accumulated squared gradients, and coordinate-wise effective learning rates. The experiment aims to illustrate how gradient history influences parameter updates and to investigate a key limitation of AdaGrad: accumulated squared gradients never decrease, so effective learning rates may become progressively smaller and eventually hinder further optimization progress [2].

2 Intuition and Core Idea

The main idea behind AdaGrad is that different model parameters may require different learning rates. During model training, the gradient indicates how the loss function changes with respect to each parameter. A large gradient suggests a strong local change in the loss, whereas a small gradient indicates a weaker local change. Using the same learning rate for every parameter may therefore produce inefficient updates when their gradient magnitudes differ considerably.

AdaGrad addresses this problem by maintaining a separate history of squared gradients for each parameter. Whenever a gradient is calculated, its square is added to the corresponding accumulated value. Parameters that repeatedly receive large gradients build up larger accumulators and consequently receive smaller effective learning rates. Parameters with smaller accumulated squared gradients retain comparatively larger effective learning rates. This mechanism allows the optimizer to adapt its updates according to the gradient history of individual parameters [1].

Figure 1 illustrates the sequence of operations performed during an AdaGrad iteration. First, the current parameter values are used to calculate the gradient of the objective function. Next, the squared gradient components are added to their respective accumulators. The accumulated values are then used to scale the gradient, producing coordinate-wise adaptive updates. Finally, the parameters are updated, and the process repeats until the chosen stopping condition is reached.

An important characteristic of AdaGrad is that its accumulated squared gradients never decrease. Consequently, the effective learning rates are non-increasing over time. Although this can be beneficial for adapting updates to different gradient scales, it may also cause learning rates to become too small during prolonged optimization.

Key Idea

AdaGrad accumulates the square of each coordinate's gradients and divides the current gradient by the square root of that coordinate's accumulated history. Coordinates with larger accumulated gradients therefore receive smaller effective step sizes.

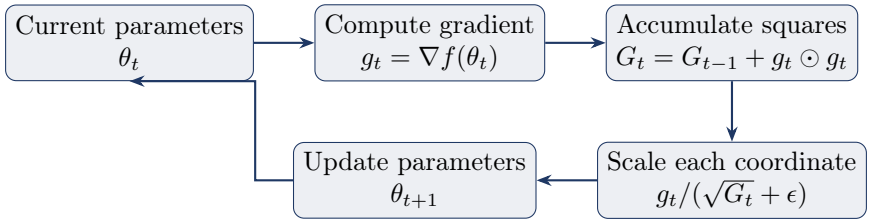


Figure 1: AdaGrad iteration: compute the gradient, accumulate its coordinate-wise square, scale the update, and repeat.

3 Mathematical Formulation

Let $\theta_t \in \mathbb{R}^d$ be the parameter vector and $g_t = \nabla f(\theta_t)$ the gradient at iteration t. AdaGrad stores the cumulative squared-gradient vector

$$G_t = G_{t-1} + g_t \odot g_t, \qquad G_{-1} = \mathbf{0}, \tag{1}$$

where $\odot$ denotes element-wise multiplication. The coordinate-wise effective learning rate is

$$\eta_{t,i}^{\text{eff}} = \frac{\eta}{\sqrt{G_{t,i}} + \epsilon}, \tag{2}$$

and the parameter update is

$$\theta_{t+1} = \theta_t - \eta \, \frac{g_t}{\sqrt{G_t} + \epsilon}. \tag{3}$$

Square roots and divisions in Equation 3 are applied element-wise. Here $\eta > 0$ is the base learning rate and $\epsilon > 0$ is a small stability constant. Implementations sometimes place ϵ inside the square root; this report uses the convention shown above.

For the visualization, use the objective

$$f(x, y) = \frac{1}{2}(x^2 + 10y^2), \qquad \nabla f(x, y) = \begin{bmatrix} x \\ 10y \end{bmatrix}. \tag{4}$$

Its Hessian is $\nabla^2 f = \operatorname{diag}(1, 10)$, which is positive definite. Thus the objective is strictly convex and has the unique minimizer $(x^\star, y^\star) = (0, 0)$. The unequal curvature makes the gradient scales different across the two coordinates, making coordinate-wise adaptation visible.

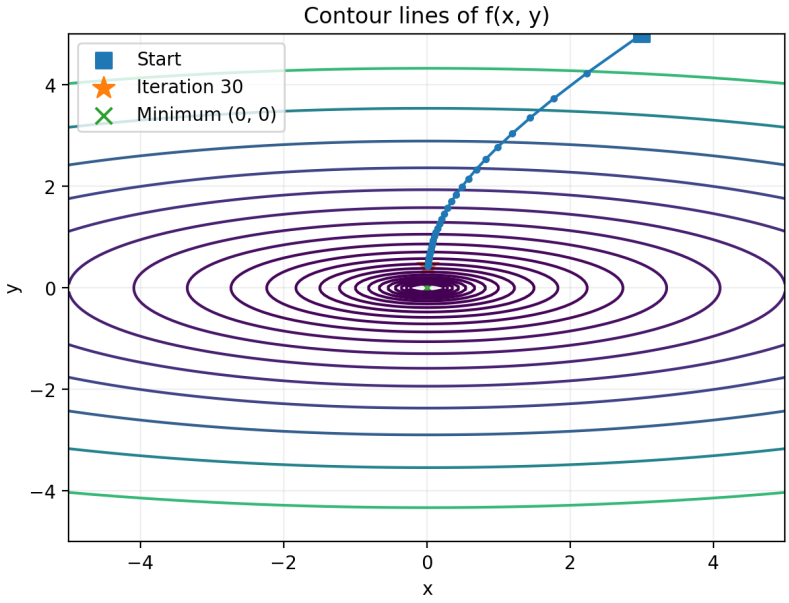


Figure 2: AdaGrad trajectory over the contour lines of the quadratic objective function.

4 Algorithm / Procedure

Algorithm 1: AdaGrad for minimizing a differentiable objective

Input: Objective f, gradient ∇f, initial parameters θ_0, learning rate η, stability constant ϵ, iteration count T

Output: Updated parameters θ_T

$G \leftarrow \mathbf{0}$; $\theta \leftarrow \theta_0$;

for $t \leftarrow 0$ **to** $T-1$ **do**

  $g \leftarrow \nabla f(\theta)$;

  $G \leftarrow G + g \odot g$;

  $\theta \leftarrow \theta - \eta\, g/(\sqrt{G} + \epsilon)$;

return θ;

Implementation convention. The code updates the accumulator with the current gradient before computing the parameter step. For a dense vector of d parameters, the accumulator and update require $O(d)$ operations and $O(d)$ extra memory per iteration, excluding gradient-computation cost.

Short implementation excerpt (the full interactive program is in the linked repository):

```
gradient = grad(theta)
accumulator += gradient ** 2
effective_lr = eta / (np.sqrt(accumulator) + epsilon)
theta = theta - effective_lr * gradient
```

5 Worked Example (Step by Step)

Consider the objective function $f(w) = w^2$, with initial parameter $w_0 = 2$ and learning rate $\eta = 0.1$. Its gradient is

$$g_t = \frac{df}{dw} = 2w_t.$$

The cumulative squared-gradient accumulator is updated as

$$G_t = G_{t-1} + g_t^2, \qquad G_{-1} = 0,$$

and the parameter update is

$$w_{t+1} = w_t - \eta \frac{g_t}{\sqrt{G_t}}.$$

For simplicity, ϵ is omitted in the hand calculations. The Python implementation includes ϵ in the denominator for numerical stability.

At iteration $t = 0$, the gradient is $g_0 = 2(2) = 4$ and the accumulator becomes $G_0 = 0+4^2 = 16$. Therefore,

$$w_1 = 2 - \frac{0.1(4)}{\sqrt{16}} = 1.9.$$

At each subsequent iteration, the new squared gradient is added to the existing accumulator. The updated accumulator determines the effective learning rate η_t^{eff} and the parameter step Δw_t:

$$\eta_t^{\text{eff}} = \frac{\eta}{\sqrt{G_t}}, \qquad \Delta w_t = \eta_t^{\text{eff}} g_t.$$

The example illustrates that the accumulated squared gradients increase at each iteration, causing the effective learning rate to decrease. Consequently, the parameter approaches the minimum at $w = 0$ through progressively smaller updates for these iterations. The example demonstrates the update mechanism but does not, by itself, establish convergence for all objective functions.

t	w_t	g_t	G_t	η_t^{eff}	w_{t+1}
0	2.0000	4.0000	16.0000	0.0250	1.9000
1	1.9000	3.8000	30.4400	0.0181	1.8311
2	1.8311	3.6623	43.8521	0.0151	1.7758
3	1.7758	3.5516	56.4663	0.0133	1.7286

Table 1: Hand calculation of AdaGrad updates for $f(w) = w^2$ with $\eta = 0.1$ and ϵ omitted. Values are rounded.

6 Animation / Software Demonstration

The AdaGrad visualizer was developed using Python, NumPy, Matplotlib, and Streamlit. It demonstrates the optimization process on the two-dimensional quadratic objective

$$f(x, y) = \frac{1}{2}(x^2 + 10y^2).$$

The application provides controls for changing the initial values of x and y, the base learning rate η, the numerical-stability constant ϵ, and the total number of iterations. An iteration slider allows the user to inspect intermediate optimization states.

The visualization displays the parameter trajectory over the loss contours, the objective value across iterations, the coordinate-wise effective learning rates, and a numerical trace containing gradients, accumulated squared gradients, and parameter updates.

How to Run. Install the dependencies listed in `requirements.txt` . Then execute `streamlit run app.py` from the animation directory. The application opens in a web browser.

Source Code. `https://github.com/Adit1N?tab=repositories`.

Key Frames. Figures 3–6 show four states captured from the running application. They illustrate the initial configuration, an early update, an intermediate iteration, and the final displayed state.

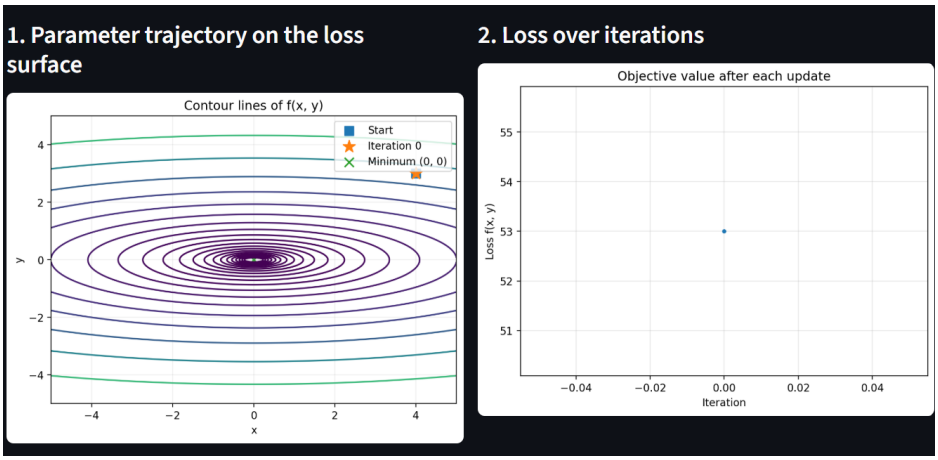


Figure 3: Initial parameter configuration.

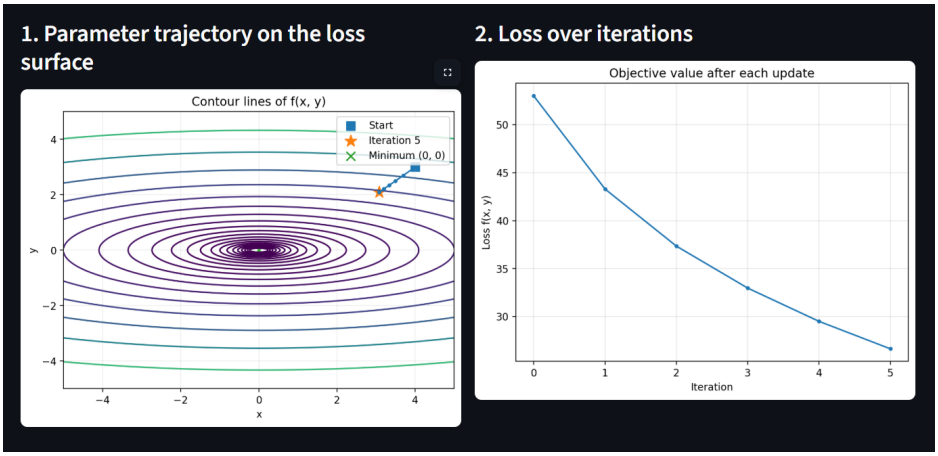


Figure 4: Early AdaGrad parameter update.

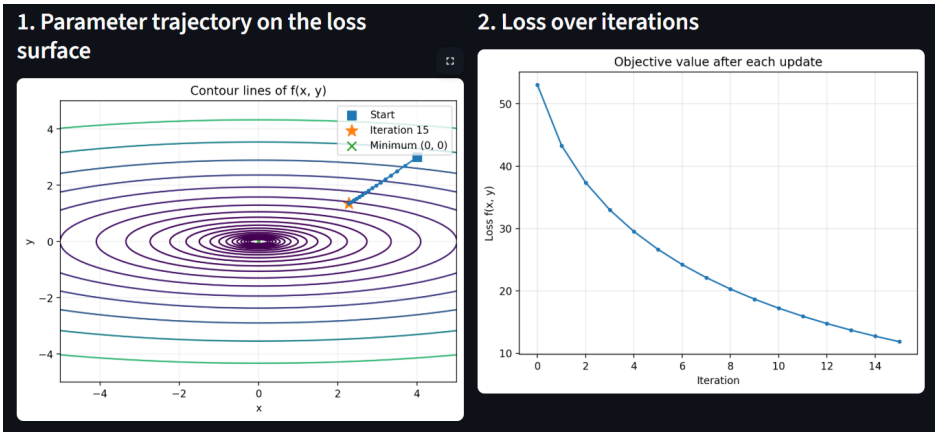


Figure 5: Intermediate optimization state.

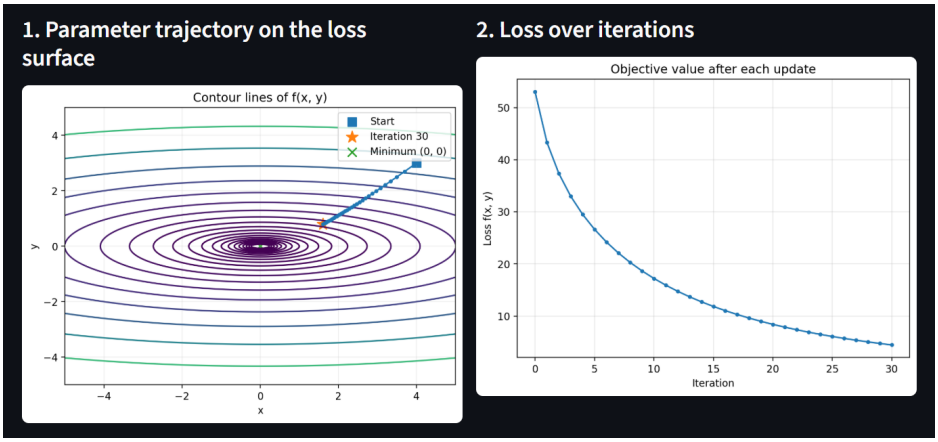


Figure 6: Final displayed optimization state.

Tool used: Python, NumPy, Matplotlib, and Streamlit.
Link: `https://github.com/your-username/adagrad-visualizer`
How to run: In the `animation` folder, run `python -m pip install -r requirements.txt`, then `streamlit run app.py`.
What the viewer can change: initial x and y, base learning rate, epsilon, total iterations, and the iteration displayed. The app plots the parameter trajectory, loss curve, effective learning rates, and numerical trace.

7 Analysis

7.1 Assumptions

The experiment minimizes the differentiable objective $f(x, y) = \frac{1}{2}(x^2 + 10y^2)$ using its analytical gradient. The initial parameters are $x = 4$ and $y = 3$, the base learning rate is $\eta = 0.30$, and the numerical-stability constant is $\epsilon = 10^{-6}$. The accumulator starts at zero. The experiment uses a deterministic gradient rather than randomly sampled mini-batches; therefore, it illustrates the adaptive update rule but does not reproduce every aspect of stochastic training.

For a dense parameter vector of dimension d, maintaining the accumulator and applying the parameter update cost $O(d)$ time and $O(d)$ additional memory per iteration. The full iteration also includes the cost of evaluating the gradient, which can dominate for large models.

Criterion	**AdaGrad**	**Plain SGD**
Learning rate	Coordinate-wise adaptive	Usually global scalar
Extra state	$O(d)$ accumulator	Often $O(1)$ optimizer state
Sparse gradients	Can be useful	May need careful tuning
Main limitation	Effective rates can become tiny	Sensitive to global rate

Table 2: Qualitative comparison of AdaGrad and plain SGD.

Strengths. AdaGrad scales each coordinate using its accumulated gradient history and can be useful when feature frequencies or gradient magnitudes differ substantially.

Limitations. Since $G_{t,i}$ never decreases, the effective learning rate for each coordinate is non-increasing under this update convention. It may become too small for useful progress late in training. Results also depend on the base learning rate, epsilon, objective, and gradient noise. The visualization of one convex function is not proof that AdaGrad is optimal for every model.

Experimental Observations

In the animation, the trajectory and loss curve show how the parameters move toward a lower-loss region. The effective learning-rate plot illustrates how the update scale changes over iterations. In this experiment, compare the initial and final losses and inspect the parameter trajectory. Since the experiment does not include a separate SGD run, it demonstrates AdaGrad's behavior but does not establish that it outperforms SGD on this objective.

8 Applications

8.1 Natural Language Processing

AdaGrad is useful in natural language processing (NLP), particularly for learning word representations and training models with sparse features. In text classification, a document may activate only a small subset of the available vocabulary features. AdaGrad adjusts the learning rate separately for each parameter according to its accumulated squared gradients, allowing parameters with relatively infrequent updates to retain larger effective learning rates. This property makes it suitable for high-dimensional, sparse learning problems [1].

8.2 Online Learning and Recommendation Systems

In online learning, models are updated as new user interactions or data samples arrive. For example, a recommendation system may update its parameters when a user clicks, rates, or purchases an item. AdaGrad adapts parameter-wise update sizes using the history of observed gradients. This can be useful when user preferences and feature frequencies vary significantly over time. However, because the accumulated squared gradients continually increase, the effective learning rates may become too small when the data distribution changes substantially.

8.3 Machine Learning with Sparse Features

AdaGrad can also be applied to classification and prediction tasks with high-dimensional sparse input features, such as spam detection and click-through-rate prediction. In spam detection, features may represent individual words or phrases, and many features occur infrequently. AdaGrad maintains a separate squared-gradient accumulator for each model parameter and scales subsequent updates accordingly. This enables different features to receive different effective learning rates without manually tuning a separate rate for every parameter [1].

8.4 Summary

These applications illustrate AdaGrad's main advantage: automatic, coordinate-wise learning-rate adaptation.

9 Conclusion

AdaGrad is an adaptive optimization algorithm that adjusts the learning rate of each parameter using the cumulative sum of its past squared gradients. This allows different parameters to receive different update sizes during optimization.

In this experiment, AdaGrad was applied to the objective function $f(x, y) = \frac{1}{2}(x^2 + 10y^2)$, starting from $(x, y) = (4, 3)$ with a base learning rate of 0.30 and $\epsilon = 10^{-6}$. The trajectory and loss plots were used to observe the optimization process. One observed result was that the loss decreased from [initial loss] to [final loss] after 30 iterations.

One limitation of this experiment is that it uses a simple, deterministic two-variable function rather than a large real-world machine-learning dataset. Therefore, the results do not establish how AdaGrad would perform on more complex learning tasks.

A meaningful direction for future work is to compare AdaGrad with plain stochastic gradient descent using the same objective function, initial parameters, and controlled learning-rate settings. This would help evaluate how adaptive learning rates affect convergence and final loss.

Declaration of AI Tool Use

- **Tools used:** ChatGPT (OpenAI).
- **What for:** ChatGPT was used to assist with understanding the AdaGrad algorithm, explaining mathematical concepts, help with Python animation code, and providing guidance on debugging and interpreting optimization plots.
- **Approximate share of AI-generated content:** [15-20]%.

I confirm that I have reviewed the submitted report and animation, verified the results I present, and taken responsibility for the final submission. The work has been completed in accordance with the course's academic integrity requirements.

Signature: **Date:**

References

[1] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *Journal of Machine Learning Research*, vol. 12, pp. 2121–2159, 2011.

[2] H. B. McMahan, "A survey of algorithms and analysis for adaptive online learning," *Journal of Machine Learning Research*, vol. 18, no. 90, pp. 1–50, 2017.